

Starting 7:05

Today's Agenda:

- Trees Intro
- Naming Convention
- Tree Traversal
- Basic Tree Problems

Arrays

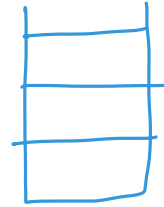


Linked list



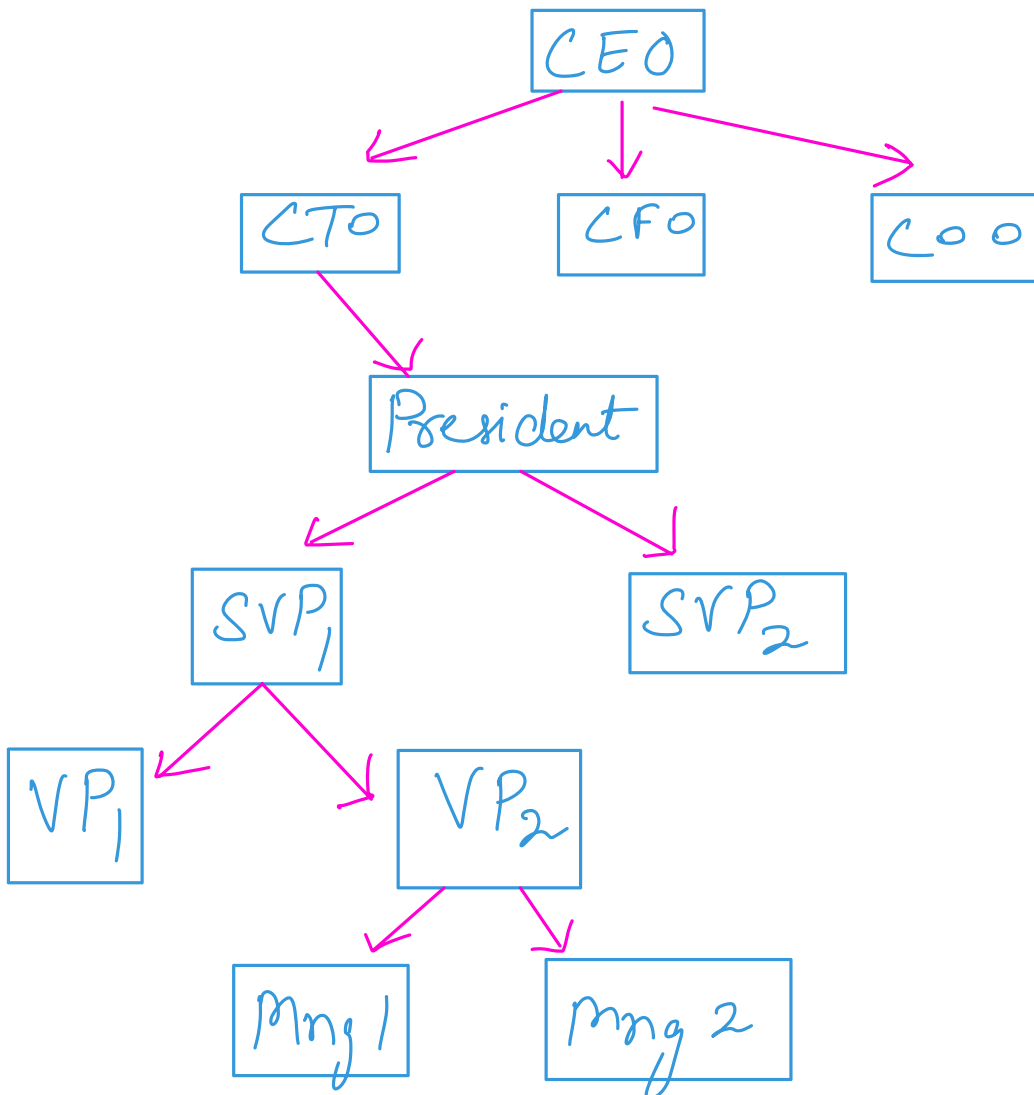
Linear

Stacks



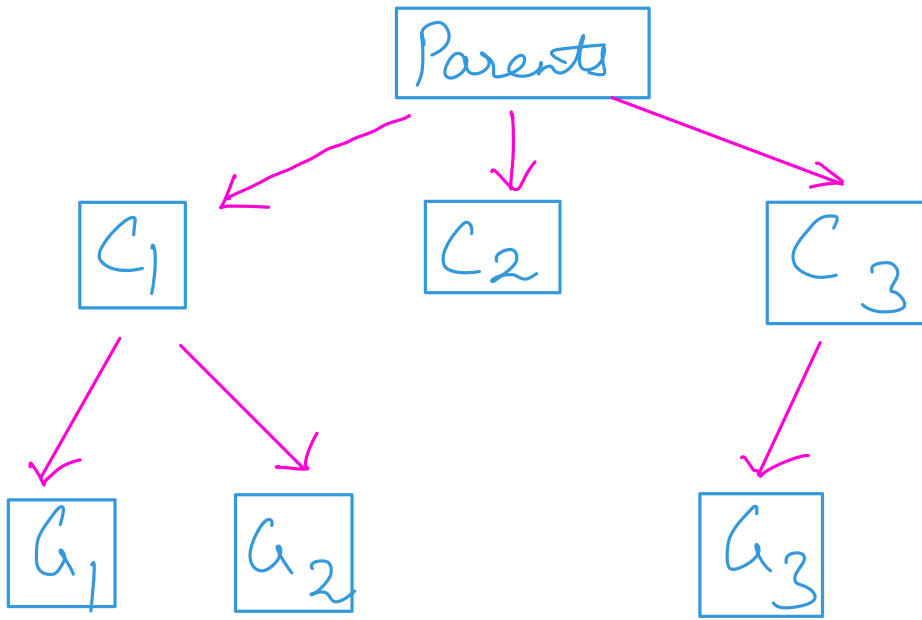
Hierarchical Data :-

Ex:- An organisation.



Ex 2:-

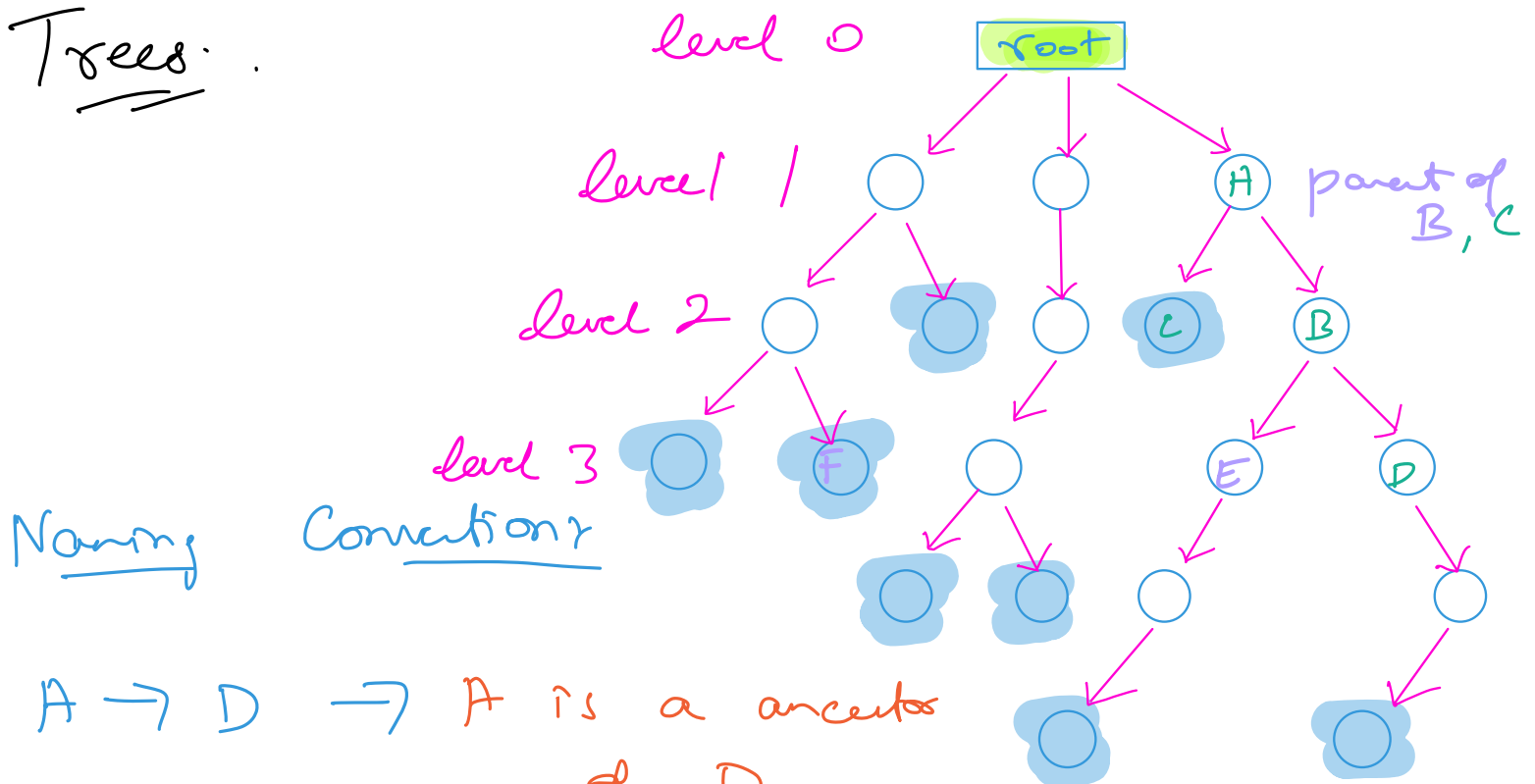
A Family Tree



Ex: 3:

Folder
Structure.

Trees:



Naming

Convention?

$A \rightarrow D \rightarrow A$ is an ancestor
of D .

$B \rightarrow C \rightarrow$ Sibling nodes.

F, E, D $\rightarrow$ Same level nodes.

root $\rightarrow$ node without a parent.

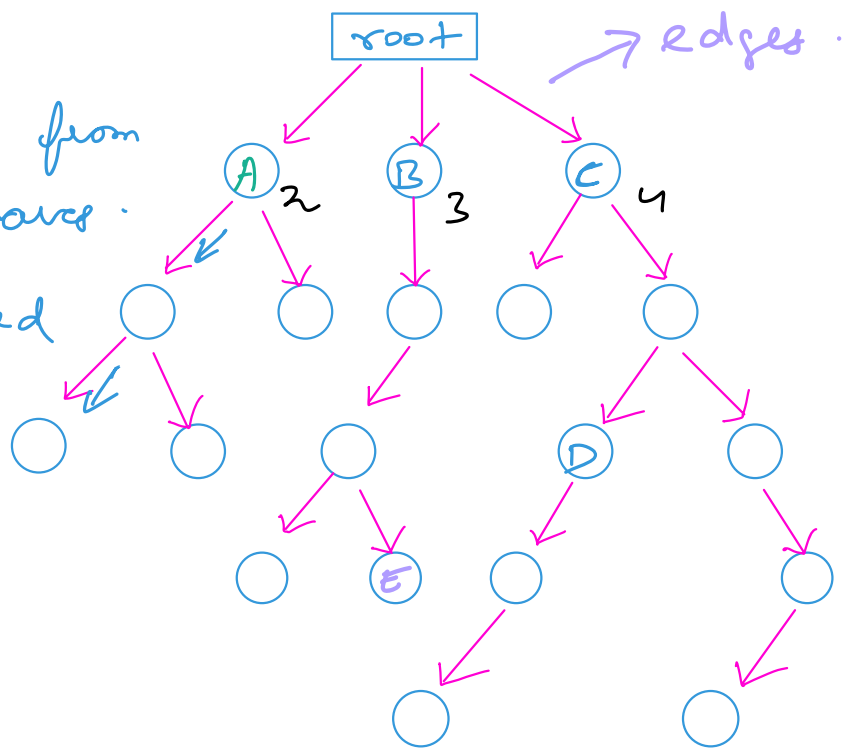
leaf $\rightarrow$ Nodes not having any child.

Tree $\rightarrow$ [Exactly 1 root node
For every node, there
will be exactly
1 parent]

Height (Node):

Length of longest path from
node to any of its ^{descendants} leaves.

Path is calculated
by no. of edges.



$$H(A) = 2.$$

$$H(B) = 3.$$

$$H(C) = 4.$$

Obs 1

$$\# H(\text{node}) = 1 + \max(\text{height of children nodes})$$

Obs 2

$$\# H(\text{leaf node}) = 0.$$

Depth (Node) :- Length of path from root to the node.

$$D(A) = 1. = D(B) = D(C)$$

$$D(E) = 4 ; \quad D(D) = 3.$$

Obs 1

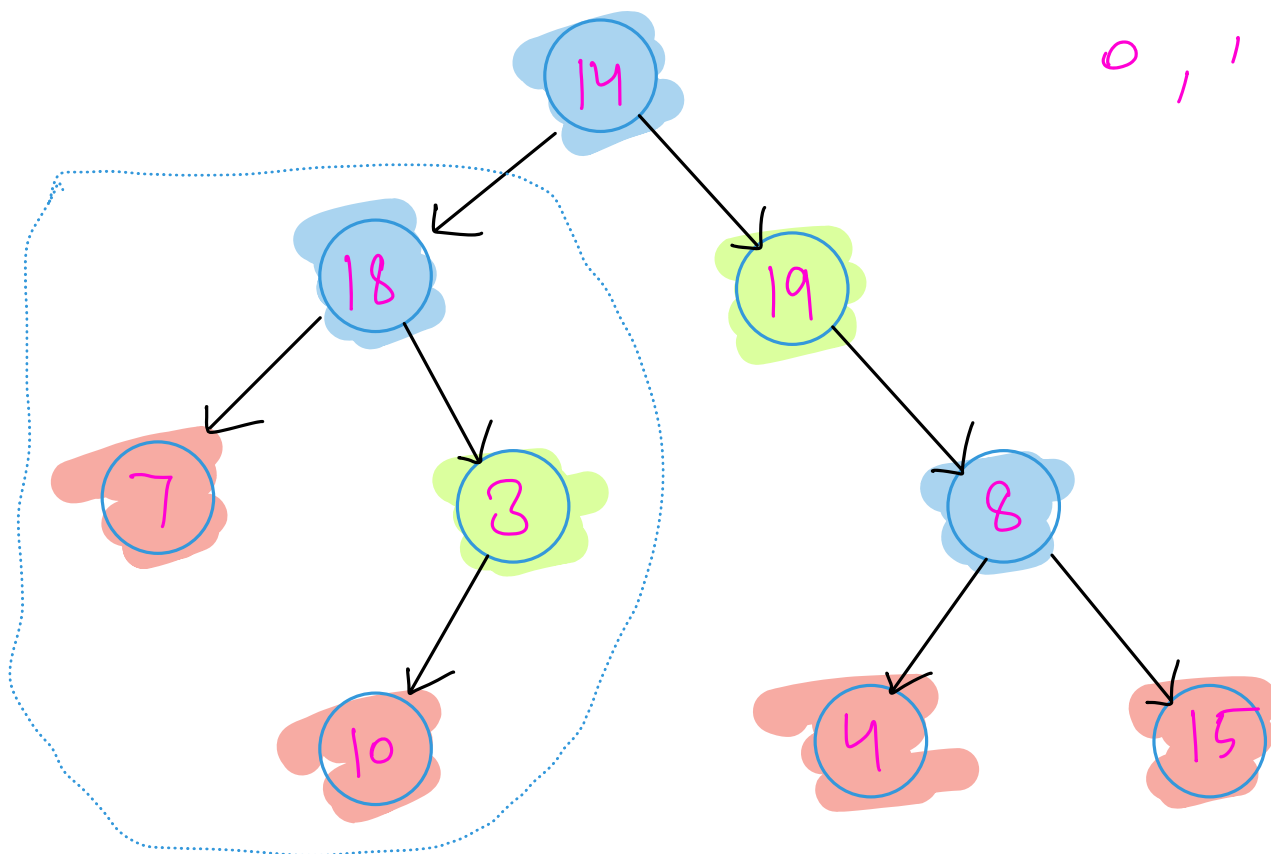
If depth of a node = D .
Then depth of child nodes
 $= D + 1.$

Obs 2

Depth of root Node = 0.

Binary Tree :

Every Node can have
at max 2 children
0, 1, 2



```
class Node {  
    int data;  
    Node left;  
    Node right;  
    Node (int x) {  
        data = x;  
        left = Null;  
        right = Null;  
    }  
}
```

Node n = new
Node(30);

#ad 1.

left #ad 2	data 30	right #ad 3
---------------	------------	----------------

#ad 3

left Null	data 50	right Null
--------------	------------	---------------

$r \rightarrow \text{left} = \text{new Node}(40);$ #ad2

left NULL	data 40	right NULL
--------------	------------	---------------

$r \rightarrow \text{right} = \text{new Node}(50);$

Constructing a tree involves a concept of deserialisation & serialisation, which will be covered in Adv. class.

For tree problems, our tree is already constructed and we are given a root node of that tree.

Traversals:

Adv. Section

$\left[\begin{array}{l} \text{Pre-order} \\ \text{Post-order} \\ \text{In-order} \end{array} \right]$

$\left[\begin{array}{l} \text{level order} \\ \text{Vertical level} \\ \text{order} \end{array} \right]$

8:10 AM

Tree Traversal:

Pre-order! DLR

Step 1: print (root → data)

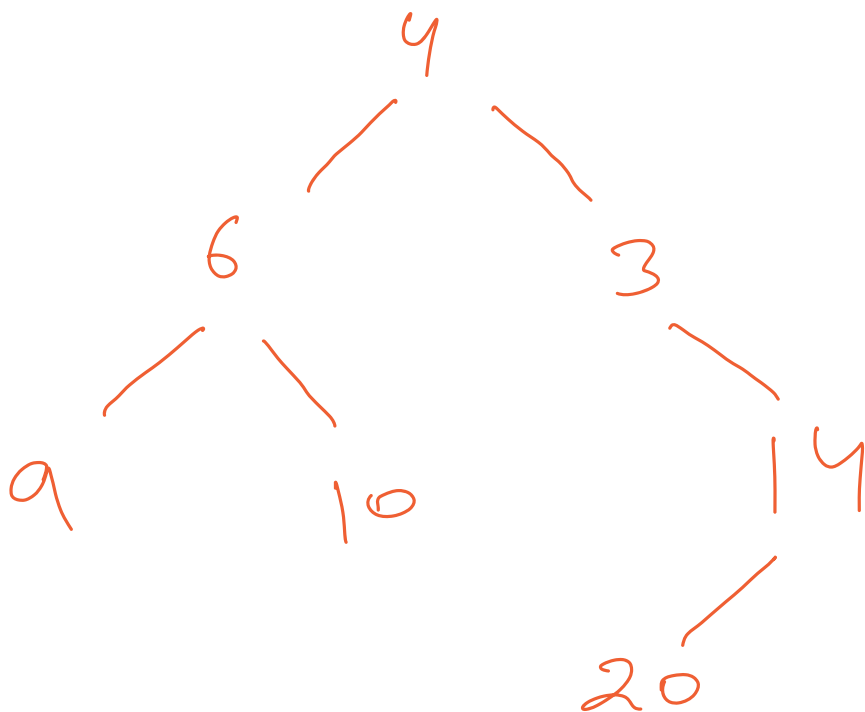
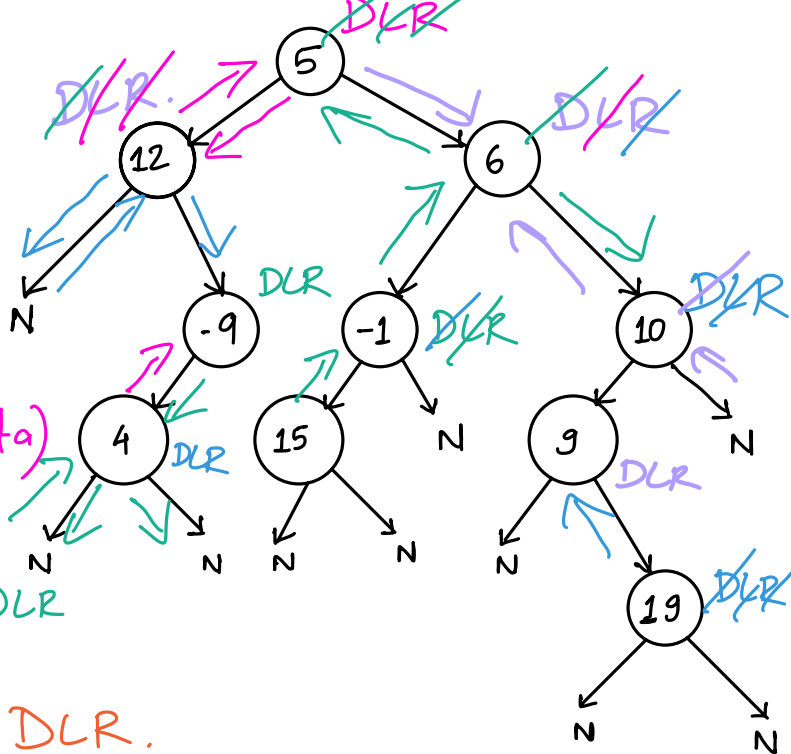
Step 2:- Go to the left subtree & print entire left subtree in DLR.

Step 3:- Go to Right Subtree & print entire RS in DLR

S.C → $O(\text{Height})$

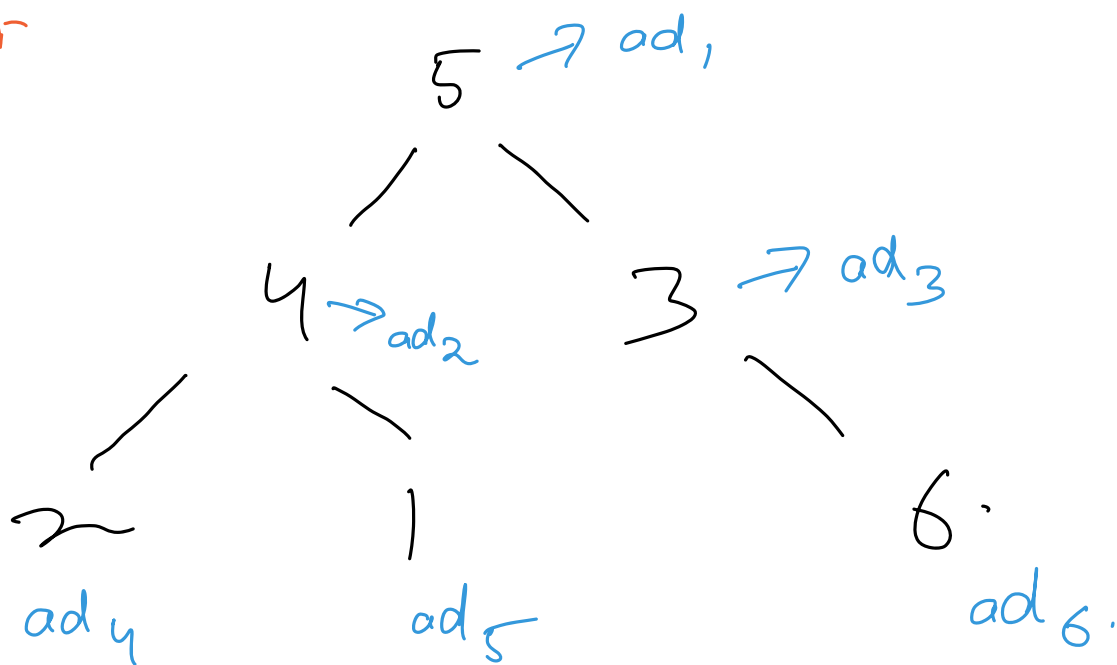
T.C → $O(N)$

Output → 5, 12, -9, 4, 6, -1, 15, 10, 9, 19



Pre-order:-
4, 6, 9, 10, 3, 14, 20

Tracing



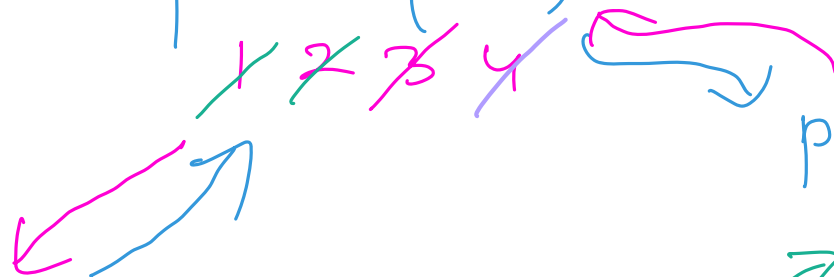
```
void preorder(Node root) {
```

```
    1 if (root == NULL) return;
    2   print (root -> data);
    3   preorder (root -> left);
    4   preorder (root -> right);
```

}

Output → 5, 4, 2, 1, 3, 6

preorder(ad1)



preorder(ad3)

1 2 3 4

pre-order (ad 2)

1 2 3 4

preorder (NULL)
1 2 3 4

pre-order (ad 5)

1 2 3 4

pre-order (ad 4)

1 2 3 4

pre-order (NULL)

1 2 3 4

preorder (NULL)

1 2 3 4

pre order (ad 5)

1 2 3 4

Pre-order $\rightarrow$ 1, 2, 3, 4.

In-order $\rightarrow$ 1, 3, 2, 4.

Main logic $\rightarrow$

inorder (root $\rightarrow$ left)

print (data)

inorder (root $\rightarrow$ right);

Post-Order $\rightarrow 1, 3, 4, 2.$

post order (root $\rightarrow$ left)
post order (root $\rightarrow$ right)
print (root $\rightarrow$ data);

Some Problems:-

① Cal. the size of the tree.
 $\rightarrow$ ⑦.

① Ass:- Given root Node, Cal. the size of tree.



② Rec. logic / Main Logic

$$\begin{aligned} \text{Size of tree} &= \text{Size of LST} \\ &+ \text{Size of RST} \\ &+ 1. \end{aligned}$$

③

```
if (root == NULL)
    return 0;
```

```
int size(Node root)
```

```
{
    if (root == NULL)
        return 0;
```

```
    int l = size(root->left);
```

```
    int r = size(root->right);
```

```
    return l + r + 1;
```

3

②

Cal. the sum of all nodes.

Ass :- Cal. the sum of all nodes.

Recursive Condition

$$\text{Sum}(\text{Tree}) = \text{Sum}(\text{LST}) + \text{Sum}(\text{RST}) + \text{Root} \rightarrow \text{data}.$$

Base Conr

if ($r == \text{NULL}$) return 0;

int sum (Node root)

{ if ($\text{root} == \text{NULL}$)
return 0;

int l = sum (root → left);

int r = sum (root → right);

return l + r + root → data;

}

③ Cal. the height of tree.

Ass :- Given the root node,
cal. the height of the tree.

Main Logic →

Height (Root) → 1 + Max (Ht of

③ Base Condition:-

HT of BST,

if (root == NULL)
 $\{ \text{return -1} \};$

int height (Node root)

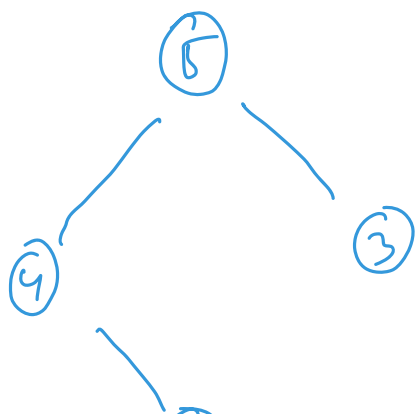
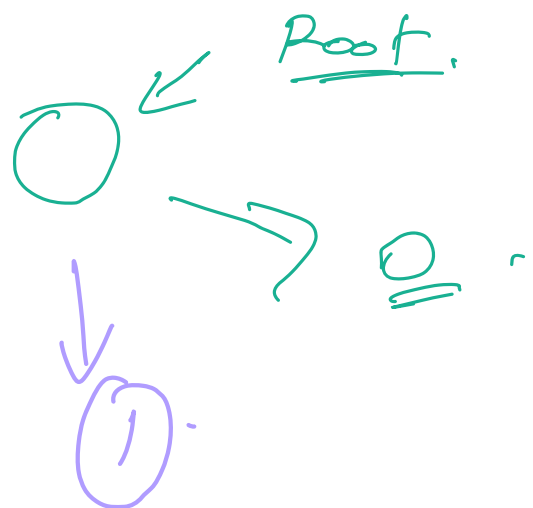
{ ① if (root == NULL)
 return -1; 0.

② int hL = height (root → L);

③ int hR = height (root → R);

④ return max(hL, hR) + 1;

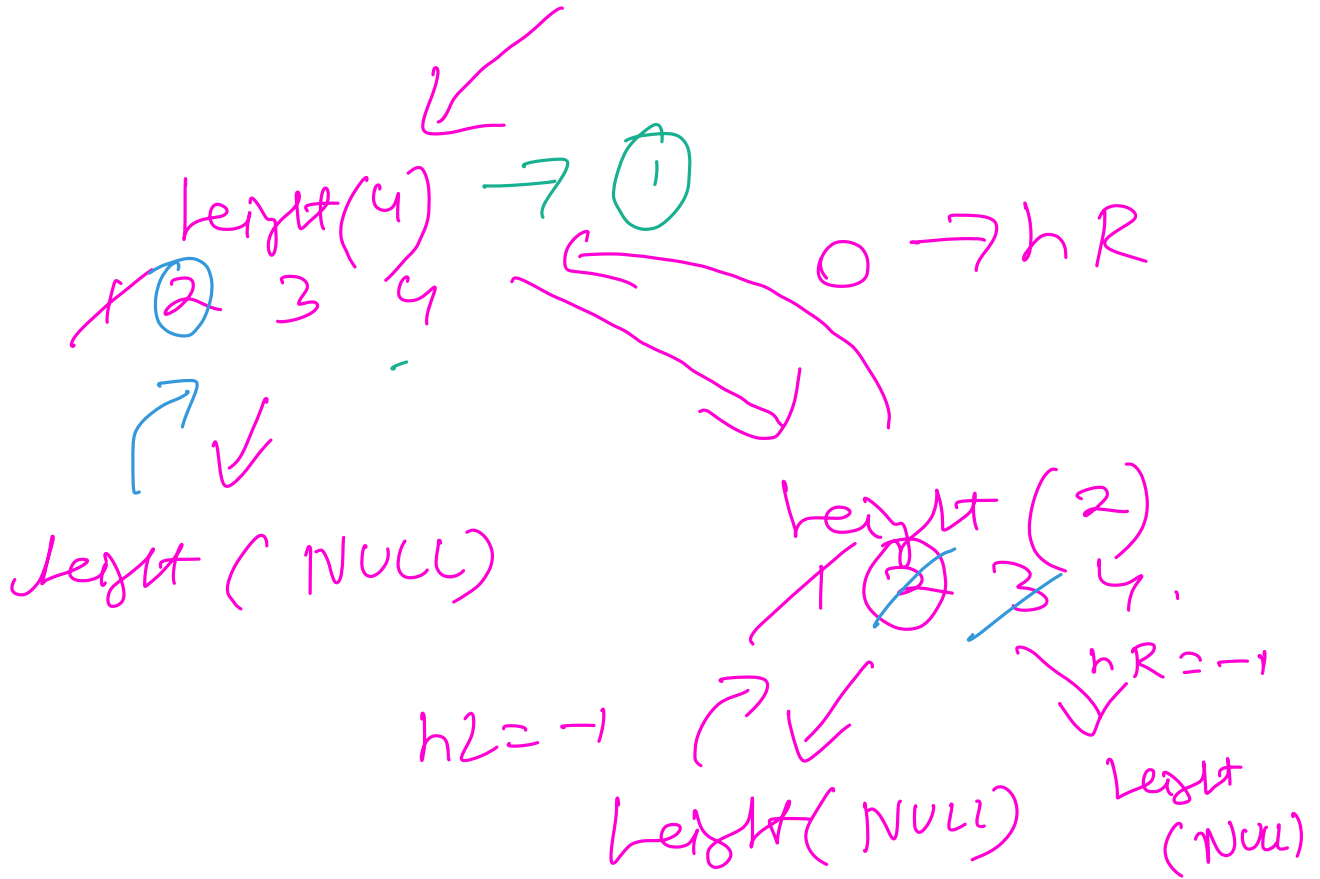
}



②

height (5)
1 ② 3 4

hL = -1



$$A -- \quad \& \quad \underline{17} = \underline{A-1};$$

54
A 77

5
A 77

